

DSA STUDY BLUEPRINT SERIES

83. Kth Level of a Binary Tree

Extracting Nodes located at Depth Level K

Section 09 • C++ Core Data Structures & Algorithms

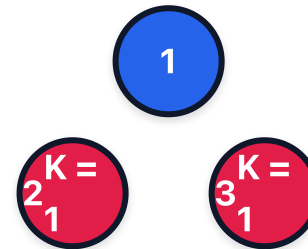
Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Print node values at depth K.
- Traverse recursively decrementing depth value until $K == 0$.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
void printKthLevel(TreeNode* root, int k) {  
    if (!root || k < 0) return;  
    if (k == 0) { cout << root->val << " "; return; }  
    printKthLevel(root->left, k - 1);  
    printKthLevel(root->right, k - 1);  
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N)$
Space Complexity	$O(H)$

Key Practices & Gotchas

- **Important:** Base case logic must stop immediately if current node is null.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace recursive Preorder Traversal visits (Node-Left-Right) for tree `1 (left=2, right=3)`:

Call Stack Level	Active Node Visited	Expected Left Traversal	Expected Right Traversal
1	Root (1)	Recurse on Left child (2)	Recurse on Right child (3)
2	Leaf (2)	Base case (nullptr)	Base case (nullptr)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Iterative traversals:** Standard DFS traversals can run iteratively by simulating recursion calls with Stacks.
- **Level Order BFS:** Level tracking searches use Queues to visit node layers.
- **Related LeetCode Problems:** #102 Binary Tree Level Order Traversal, #94 Binary Tree Inorder Traversal, #236 Lowest Common Ancestor.